AI Bootcamp Week 3: Model Invocation, Adversarial Agents, and Bounded Agents

Tuesday — Student Exercise Guide

© 2026 Improving

OBJECTIVES

By end of Week 3, the candidate can:

- Explain the DLP automation spectrum and what evidence is required to move up each step
- Explain why the DLP defaults to skills with `disable-model-invocation: true`, and articulate the three graduation criteria for enabling model invocation
- Write invocation test cases (positive trigger, negative trigger, quality) and use them to make a documented enable-or-gap decision on a real skill
- Build an adversarial agent that challenges a skill's output, and explain how it complements (not duplicates) a Python evaluation script
- Run an evaluator-vs-adversarial comparison and use the delta to improve either the EC or the skill
- Compose skills into a bounded agent: a multi-step pipeline run in one IDE session with evaluation and adversarial review at key steps
- Explain context window degradation, why bounded agents hit a ceiling, and what a custom harness is (awareness level; building one is Stage 4 work)
- Describe the IIOS stage framework: where DLP Completion sits (Stage 3, Task) and what Stage 4 (Workflow) requires

© 2026 Improving

TIMING

- **Monday:** 1:00 PM – 5:00 PM (travel morning: 4 hours)
- **Tuesday–Thursday:** 9:00 AM – 5:00 PM with 1-hour lunch (7 hours each)
- **Friday:** 9:00 AM – 12:00 PM (travel afternoon: 3 hours)

© 2026 Improving

TUESDAY (9:00 AM – 5:00 PM): ENABLING MODEL INVOCATION

9:00 – 10:30 AM: EDD FOR INVOCATION: THE THREE CASE TYPES

Skills with invocation enabled require a different kind of test than output quality alone. You need all three case types:

- **Positive trigger cases:** input scenarios where the skill SHOULD invoke. Expected: the model invokes it, and the output meets the specification.
- **Negative trigger cases:** input scenarios where the skill SHOULD NOT invoke. Expected: the model handles the input another way and the skill stays silent.
- **Quality cases:** given that the skill did invoke, does the output meet the EC you have been evaluating all along?

Target: 3 verified positive trigger cases + 2 quality cases, plus negative cases for the boundaries you are least sure about. Write the test file before flipping any flag.

10:30 AM – 12:00 PM: THE GRADUATION ATTEMPT

Pick your most-iterated skill: the one with the most evaluation history from Weeks 1 and 2. Apply the full graduation check:

1. Write the three graduation criteria (trigger inputs, output specification, 3 verified trigger test cases)
2. Confirm the EDD prerequisite: 3+ documented iteration cycles
3. Write the invocation test file: positive, negative, quality cases
4. Flip `disable-model-invocation` to `false` and verify: set up each trigger context, confirm the model actually invokes the skill, check the output against spec. Then run the negative cases and confirm it stays silent.

12:00 – 1:00 PM: LUNCH

1:00 – 3:00 PM: BOTH OUTCOMES ARE VALID

You leave today with one of two artifacts:

- **Enabled:** the skill has model invocation enabled, with the graduation criteria and invocation test cases committed alongside it. Note the test cases so they can be re-validated later.
- **Gap document:** the skill stays at `disable-model-invocation: true`, and you have committed a written specification of exactly which criterion is not met and what evidence would close the gap.

Both are passing outcomes. What is not acceptable: vague "it is not ready" with no specific gap named. An unnamed gap means you have not looked.

If you enabled invocation, treat the rest of the week as a validation period. Did the skill invoke when it should? Did it stay silent when it should not? Address any over-invocation immediately, before it becomes a habit.

© 2026 Improving

3:00 – 5:00 PM: BOUNDARY AMBIGUITY WORKSHOP

In pairs, examine skill configurations where the invocation boundary is genuinely ambiguous. For each:

- Where would the model likely disagree with a human about whether to invoke?
- Is the failure mode over-invocation or under-invocation?
- What change to the skill's description or frontmatter would reduce the ambiguity?

Then simulate the team scenario: two engineers have independently enabled invocation on their own versions of the same skill, and the versions disagree about when to fire. Walk the alignment process: which trigger test cases does each version pass and fail on the other's test file? This is why the DLP treats invocation as a team-level, evidence-backed decision rather than an individual convenience.

© 2026 Improving